



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Ariel Adara Mercado Martínez

Asignatura: Estructura de Datos y Algoritmos I

Grupo: 5

No de Práctica(s): Reporte del programa "Postfija"

Integrante(s): Chavez Sanchez Ehidan Javier

*No. de lista o
brigada:*

5

Semestre: 2026-2

Fecha de entrega: 17 de Mayo del 2026

Observaciones:

CALIFICACIÓN: _____

Introducción:

El presente reporte relata y describe la implementación de tipos de datos abstractos (TDA) en lenguaje de programación C. Básicamente un sistema de conversión y evaluación de expresiones algebraicas, desarrollado como proyecto en la materia de Estructura de datos y Algoritmos I.

El programa lee desde un archivo de texto las definiciones de variables y una expresión algebraica escrita en notación infija, que es la forma convencional en la que se escriben las expresiones matemáticas, por ejemplo $(a+b)*c$. A partir de ella, el sistema realiza la conversión a notación postfija, también conocida como Notación Polaca Inversa, donde los operadores se colocan después de sus operandos, resultando en algo como $a\ b\ +\ c\ *$. Finalmente evalúa la expresión y muestra el resultado numérico.

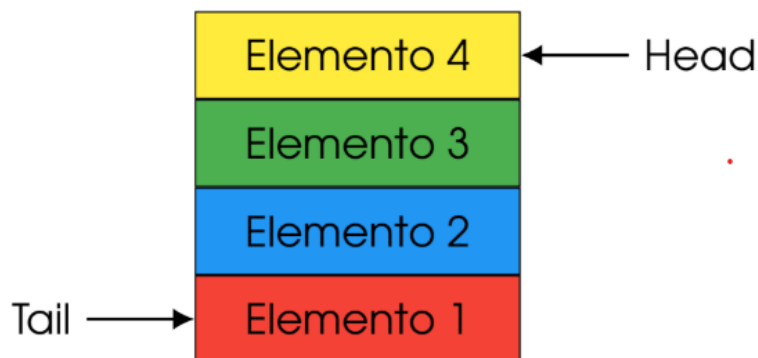
Para lograr esto, el programa hace uso de dos estructuras de datos dinámicas implementadas con listas enlazadas: una pila, que sigue la política LIFO (último en entrar, primero en salir), utilizada tanto en el proceso de conversión mediante el algoritmo como en la evaluación de la expresión postfija; y una cola, que sigue la política FIFO (primero en entrar, primero en salir), utilizada para almacenar y preservar el orden de los tokens de la expresión postfija generada.

El sistema soporta los operadores de suma, resta, multiplicación, división y potencia, respetando su precedencia y asociatividad correspondiente.

Descripción de estructuras utilizadas.

Pila:

La pila es un TDA lineal que sigue una secuencia LIFO (Last in, First out), o por su traducción “Ultimo elemento en entrar, primer elemento en Salir”. En lenguaje C, el TDA Pila se construye combinando structs para definir sus límites y elementos, y funciones dedicadas para manipular su tope. Gráficamente se puede representar a la pila de la siguiente manera:



En donde el tail representa siempre y sin excepción el ultimo elemento del conjunto de datos, y por su contraparte el head o cabeza representa la cima o el ultimo elemento del conjunto de datos.

Cualquier implementación de una pila debe cumplir con la especificación de sus funciones principales:

crearPila: Se reserva la memoria para un puntero dirigido hacia el TDA “pila”, esto a través de malloc, siempre se debe verificar que el puntero no sea ==NULL.

esVacia: Comprueba si hay elementos.

Push: Inserta un elemento en la cima del conjunto de datos.

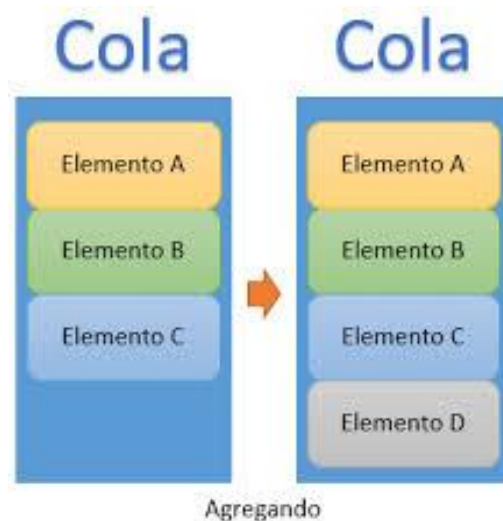
Pop: Saca el elemento de la cima del conjunto de datos y lo destruye.

Peek: Se dirige a través de ciclos a la cima del conjunto de datos y lo imprime en el Shell.

En conclusión, El TDA “pila” permite gestionar una colección de datos donde solo importa el último elemento procesado posibilitando acciones como: Acceder a información restringida o manipulación y control de estados.

Cola:

Una cola es un Tipo de Dato Abstracto (TDA) que organiza los elementos bajo la política FIFO (First In, First Out), lo que significa que el primer elemento en ingresar es el primero en salir. Gráficamente es muy similar a la pila, sin embargo se puede analizar de la siguiente manera:



Cualquier implementación de un TDA “Cola” debe cumplir con la especificación de sus funciones principales:

- **crearCola:** Reserva memoria para la estructura de la cola e inicializa sus punteros en NULL.
- **estaVacia:** Comprueba si el puntero del frente es NULL para saber si hay elementos
- **encolar:** Crea un nodo nuevo con el valor y lo coloca al final de la cola
- **desencolar:** Elimina el nodo del frente, avanza la cola y libera memoria
- **ontenerFrente:** devuelve el valor

Uso de memoria dinámica:

La memoria dinámica en C permite reservar y liberar memoria mientras el programa está en ejecución, en lugar de definir un tamaño fijo desde el inicio. Esto es útil cuando no se sabe de antemano cuántos elementos se van a manejar, como en el caso de las pilas y colas de este proyecto, donde los nodos se crean con malloc cada vez que se inserta un elemento y se liberan con free cada vez que se extrae uno. Así la estructura crece y decrece según se necesite, sin desperdiciar memoria.



Algoritmo de conversión:

Precedencia:

En nuestro programa la precedencia es la que define que operador se evalúa primero cuando hay varios en una expresión postfija, se implementó a manera de que el operador de potencia (^) tiene mayor prioridad, en segunda posición el operador de la multiplicación (*) y división (/), y finalmente la suma (+) y resta (-)

adquiriendo la menor prioridad, esto garantiza que expresiones como $a+b*c$ se evalúen correctamente $a+(b*c)$.

```
a = 5
b = 3
c = 2

(a+b)*c
```

Ejemplo en compilación.

Asociatividad:

La asociatividad define en pocas palabras el orden de evaluación cuando dos operadores tienen la misma precedencia. La mayoría de los operadores son asociativos por la izquierda, lo que significa que se evalúan de izquierda a derecha.

Funcionamiento del algoritmo infija-postfija:

Este algoritmo recorre la expresión carácter por carácter y aplica las siguientes reglas: si el token es un operando, se envía directamente a la cola de salida; si es un paréntesis izquierdo, se apila; si es un paréntesis derecho, se vacían operadores de la pila hacia la cola hasta encontrar el paréntesis izquierdo correspondiente; y si es un operador, se desapilan hacia la cola todos los operadores del tope que tengan mayor precedencia o igual precedencia con asociatividad izquierda, y luego se apila el operador actual. Al terminar de leer la expresión, se vacían todos los operadores restantes de la pila hacia la cola.

Evaluación postfija

Para evaluar la expresión postfija se utiliza una pila de valores numéricos. El algoritmo recorre la cola postfija valor por valor: si encuentra un operando, busca su valor en la tabla de variables y lo apila; si encuentra un operador, desapila dos valores, aplica la operación correspondiente y apila el resultado. Al finalizar, el único valor que queda en la pila es el resultado de la expresión. Por ejemplo, para la expresión $a\ b\ +\ c\ *\$ con $a=5$, $b=3$ y $c=2$, primero se apilan 5 y 3, se suman dando 8, luego se apila 2, se multiplica dando 16 como resultado final.

Casos Prueba:

Caso 1:

-Archivo de entrada

```
a = 2
b = 3
c = 4

a+b*c
```

-Salida en el Shell

```
Expresion infija:
a+b*c

Expresion postfija:
a b c * +

Resultado:
14.00
```

Caso 2:

-Archivo de entrada

```
a = 5
b = 1
c = 2

(a+b)*c
```

-Salida en el Shell

```
Expresion infija:
(a+b)*c

Expresion postfija:
a b + c *

Resultado:
12.00
```

Explicación del código implementado:

En primera instancia se analizaron los archivos.h propuestos en github por la docente en la carpeta "include"



Cada archivo implementaba la estructura de los archivos .c, ya que en ellos se contenían las firmas de las funciones con las que se trabajaría en la carpeta SRC.

Pila.c

pila.c contiene la implementación de la pila, que es una estructura que funciona como una pila de platos: el último que pones es el primero que puedes quitar.

crearPila reserva memoria para una pila nueva y la deja vacía apuntando a NULL. pilaVacía simplemente revisa si el tope apunta a NULL, es decir si no hay nada apilado. push agrega un nuevo elemento hasta arriba de la pila, reservando memoria para el nodo y copiando el dato con memcpy. pop saca el elemento de hasta arriba, guarda el dato, libera la memoria del nodo y lo retorna. peek solo espía el elemento de hasta arriba sin sacarlo, útil para comparar operadores sin modificar la pila. Finalmente destruirPila recorre todos los nodos liberando su memoria uno por uno hasta dejar la pila vacía.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #include "../include/pila.h"
6
7  Pila* crearPila() {
8      Pila *p = (Pila *)malloc(sizeof(Pila));
9      if (p == NULL) {
10         printf("Error: sin memoria para crear pila.\n");
11         return NULL;
12     }
13     p->tope = NULL;
14     return p;
15 }
16
17 // Se reserva memoria para el TODA "pila" y se verifica que
18
19 int pilaVacía(Pila *p) {
20     return p->tope == NULL;
21 }
22
23 void push(Pila *p, void *dato, size_t size) {
24     NodoPila *nuevo = (NodoPila *)malloc(sizeof(NodoPila));
25     if (nuevo == NULL) {
26         printf("Error: sin memoria para push.\n");
27         return;
28     }
29     nuevo->dato = malloc(size);
30     if (nuevo->dato == NULL) {
31         printf("Error: sin memoria para dato en push.\n");
32         return;
33     }
34     memcpy(nuevo->dato, dato, size);
35     nuevo->siguiente = p->tope;
36     p->tope = nuevo;
37 }

```

```

38
39 // Ingresa un nuevo elemento en la cima de la pila
40
41 char pop(Pila *p) {
42     if (pilaVacía(p)) {
43         printf("Error: pop en pila vacía.\n");
44         return '\0';
45     }
46     NodoPila *temp = p->tope;
47     p->tope = temp->siguiente;
48     char dato = *((char *)temp->dato);
49     free(temp->dato);
50     free(temp);
51     return dato;
52 }
53
54 char peek(Pila *p) {
55     if (pilaVacía(p)) return '\0';
56     return *((char *)p->tope->dato);
57 }
58
59 void destruirPila(Pila *p) {
60     while (!pilaVacía(p)) {
61         NodoPila *temp = p->tope;
62         p->tope = temp->siguiente;
63         free(temp->dato);
64         free(temp);
65     }
66     free(p);
67 }

```

Cola.c

cola.c contiene la implementación de la cola, que a diferencia de la pila funciona como una fila del banco: el primero que llega es el primero que sale.

crearCola reserva memoria para una cola nueva y deja tanto el frente como el final apuntando a NULL. colaVacía revisa si el frente apunta a NULL, lo que significa que no hay nadie en la fila. enqueue agrega un nuevo elemento al final de la cola, reservando memoria para el nodo y copiando el dato, si la cola estaba vacía el frente y el final apuntan al mismo nodo, si no, el último nodo actual enlaza al nuevo. dequeue saca el elemento del frente, guarda el dato, libera la memoria y avanza el frente al siguiente nodo. imprimirCola recorre todos los nodos sin modificar la cola e imprime cada elemento separado por espacios, que es como se muestra la expresión postfija. Finalmente destruirCola libera todos los nodos y la estructura completa.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #include "../include/cola.h"
6
7  Cola* crearCola() {
8      Cola *c = (Cola *)malloc(sizeof(Cola));
9      if (c == NULL) {
10         printf("Error: sin memoria para crear cola.\n");
11         return NULL;
12     }
13     c->frente = NULL;
14     c->final = NULL;
15     return c;
16 }
17
18 // Asigna memoria al puntero de la cola c
19
20 int colaVacía(Cola *c) {
21     return c->frente == NULL;
22 }
23
24 // Revisa si el frente de la pila apunta a NULL
25
26 void enqueue(Cola *c, void *dato, size_t size) {
27     NodoCola *nuevo = (NodoCola *)malloc(sizeof(NodoCola));
28     if (nuevo == NULL) {
29         printf("Error: sin memoria para enqueue.\n");
30         return;
31     }
32     nuevo->dato = malloc(size);
33     if (nuevo->dato == NULL) {
34         printf("Error: sin memoria para dato en enqueue.\n");
35         return;
36     }
37     memcpy(nuevo->dato, dato, size);
38     nuevo->siguiente = NULL;
39
40     if (colaVacía(c)) {
41         c->frente = nuevo;
42         c->final = nuevo;
43     } else {
```

```
44         c->final->siguiente = nuevo;
45         c->final = nuevo;
46     }
47 }
48
49 char dequeue(Cola *c) {
50     if (colaVacía(c)) {
51         printf("Error: dequeue en cola vacía.\n");
52         return '\0';
53     }
54     NodoCola *temp = c->frente;
55     c->frente = temp->siguiente;
56     if (c->frente == NULL)
57         c->final = NULL;
58
59     char dato = *((char *)temp->dato);
60     free(temp->dato);
61     free(temp);
62     return dato;
63 }
64
65 void imprimirCola(Cola *c) {
66     NodoCola *actual = c->frente;
67     while (actual != NULL) {
68         printf("%c", *((char *)actual->dato));
69         if (actual->siguiente != NULL)
70             printf(" ");
71         actual = actual->siguiente;
72     }
73     printf("\n");
74 }
75
76 void destruirCola(Cola *c) {
77     while (!colaVacía(c)) {
78         NodoCola *temp = c->frente;
79         c->frente = temp->siguiente;
80         free(temp->dato);
81         free(temp);
82     }
83     free(c);
84 }
```


Parser.c:

parser.c contiene tres partes principales.

La primera es cargarArchivo, que se encarga de abrir y leer el archivo de entrada línea por línea usando fgets. Para cada línea revisa si contiene un signo =, si es así la trata como una definición de variable y usa strtok para separar el nombre del valor y guardarlos en el arreglo de variables. Si la línea no tiene = y no está vacía, la trata como la expresión algebraica y la copia directamente.

La segunda parte son las funciones auxiliares esOperador y precedencia, que simplemente indican si un carácter es un operador válido y qué nivel de prioridad tiene, siendo ^ el de mayor prioridad, luego * y /, y finalmente + y -.

La tercera y más importante es infijaAPostfija, que implementa el algoritmo Shunting-Yard. Recorre la expresión carácter por carácter y va tomando decisiones: si encuentra una letra la manda directo a la cola de salida, si encuentra un paréntesis izquierdo lo apila, si encuentra un paréntesis derecho vacía la pila hacia la cola hasta encontrar su pareja, y si encuentra un operador saca de la pila todos los que tengan mayor o igual precedencia antes de apilarlo, respetando que ^ es asociativo por la derecha. Al terminar vacía todo lo que quedó en la pila hacia la cola, dejando la expresión en orden postfijo lista para evaluarse.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #include "../include/parser.h"
6  #include "../include/pila.h"
7  #include "../include/cola.h"
8
9  void cargarArchivo(
10     const char* nombreArchivo,
11     Variable variables[],
12     int* totalVariables,
13     char expresion[]
14 ) {
15     FILE *arch = fopen(nombreArchivo, "r");
16     if (arch == NULL) {
17         printf("yo creo que no se va a poder %s.\n", nombreArchivo);
18         return;
19     }
20
21     char linea[MAX_EXPR];
22     *totalVariables = 0;
23
24     while (fgets(linea, sizeof(linea), arch) != NULL) {
25         int longitud = strlen(linea);
26         if (longitud > 0 && linea[longitud - 1] == '\n') {
27             linea[longitud - 1] = '\0';
28         }
29
30         if (strlen(linea) == 0) continue;
31
32         int igual = 0;
33         for (int i = 0; i < strlen(linea); i++) {
34             if (linea[i] == '=') {
35                 igual = 1;
36                 break;
37             }
38         }
39
40         if (igual == 1) {
41             char *nombre = strtok(linea, " =");
42             char *numtext = strtok(NULL, " =");
43
44             variables[*totalVariables].nombre = nombre[0];
45             variables[*totalVariables].valor = atof(numtext);
46             *totalVariables = *totalVariables + 1;
47         } else {
```

```

48     strcpy(expresion, linea);
49     }
50     }
51
52     fclose(arch);
53 }
54
55 int esOperador(char c) {
56     if (c == '+' || c == '-' || c == '*' || c == '/' || c == '^') {
57         return 1;
58     }
59     return 0;
60 }
61
62 int precedencia(char op) {
63     if (op == '^') {
64         return 3;
65     } else if (op == '*' || op == '/') {
66         return 2;
67     } else if (op == '+' || op == '-') {
68         return 1;
69     }
70     return 0;
71 }
72
73 Cola* infijaAPostfija(char* expresion) {
74     Cola *postfija = crearCola();
75     Pila *pila = crearPila();
76
77     int longitud = strlen(expresion);
78
79     for (int i = 0; i < longitud; i++) {
80         char actual = expresion[i];
81
82         if (actual == ' ') {
83             continue;
84         }
85
86         if ((actual >= 'a' && actual <= 'z') || (actual >= 'A' && actual <= 'Z')) {
87             enqueue(postfija, &actual, sizeof(char));
88         }
89         else if (actual == '(') {
90             push(pila, &actual, sizeof(char));
91         }
92         else if (actual == ')') {
93             while (pilaVacía(pila) == 0 && peek(pila) != '(') {
94                 char op = pop(pila);

```

```

95             enqueue(postfija, &op, sizeof(char));
96         }
97         if (pilaVacía(pila) == 0) {
98             pop(pila);
99         }
100     }
101     else if (esOperador(actual) == 1) {
102         while (pilaVacía(pila) == 0 && peek(pila) != '(') {
103             char tope = peek(pila);
104             int ant = precedencia(actual);
105             int precTope = precedencia(tope);
106
107             int sacar = 0;
108
109             if (actual != '^' && precTope >= ant) {
110                 sacar = 1;
111             } else if (actual == '^' && precTope > ant) {
112                 sacar = 1;
113             }
114
115             if (sacar == 1) {
116                 char op = pop(pila);
117                 enqueue(postfija, &op, sizeof(char));
118             } else {
119                 break;
120             }
121         }
122         push(pila, &actual, sizeof(char));
123     }
124 }
125
126 while (pilaVacía(pila) == 0) {
127     char op = pop(pila);
128     enqueue(postfija, &op, sizeof(char));
129 }
130
131 destruirPila(pila);
132 return postfija;
133 }

```

Evaluator.c:

evaluator.c contiene dos funciones.

La primera es obtenerValorVariable, que recibe el arreglo de variables y un nombre de carácter, lo busca recorriendo el arreglo uno por uno y retorna su valor numérico. Si no lo encuentra imprime un mensaje de error y retorna cero.

La segunda y principal es evaluarPostfija, que recibe la cola con la expresión postfija ya generada y la tabla de variables, y se encarga de calcular el resultado final. Para esto crea una pila de valores numéricos y recorre la cola token por token. Si el token es una letra busca su valor en la tabla y lo apila. Si el token es un operador saca los dos valores que están hasta arriba de la pila, siendo el primero en salir el segundo operando y el segundo en salir el primero, aplica la operación correspondiente con un switch y apila el resultado. Para la potencia en lugar de usar la función pow de la librería matemática se optó por un ciclo que multiplica el número por sí mismo la cantidad de veces indicada. Al terminar de recorrer la cola, el único valor que queda en la pila es el resultado final de la expresión, que se extrae y se retorna.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #include "../include/evaluator.h"
5  #include "../include/pila.h"
6  #include "../include/cola.h"
7
8  float obtenerValorVariable(
9      Variable variables[],
10     int total,
11     char nombre
12 ) {
13     for (int i = 0; i < total; i++) {
14         if (variables[i].nombre == nombre) {
15             return variables[i].valor;
16         }
17     }
18     printf("Error mi chav@ la variable '%c' no esta definida.\n", nombre);
19     return 0.0;
20 }
21
22 float evaluarPostfija(
23     Cola* postfija,
24     Variable variables[],
25     int totalVariables
26 ) {
27     Pila* pilaNum = crearPila();
28
29     while (colaVacia(postfija) == 0) {
30         char actual = dequeue(postfija);
31
32         if ((actual >= 'a' && actual <= 'z') || (actual >= 'A' && actual <= 'Z')) {
33             float valor = obtenerValorVariable(variables, totalVariables, actual);
34             push(pilaNum, &valor, sizeof(float));
35         }
36         else if (actual == '+' || actual == '-' || actual == '*' || actual == '/' || actual == '^') {
37
38             float b = 0.0;
39             if (pilaVacia(pilaNum) == 0) {
40                 b = *((float*)pilaNum->tope->dato);
```

```

41         pop(pilaNum);
42     }
43
44     float a = 0.0;
45     if (pilaVacia(pilaNum) == 0) {
46         a = *((float*)pilaNum->tope->dato); // bien
47         pop(pilaNum);
48     }
49
50     float res = 0.0;
51
52     switch (actual) {
53         case '+': res = a + b; break;
54         case '-': res = a - b; break;
55         case '*': res = a * b; break;
56         case '/':
57             if (b == 0.0) {
58                 printf("Como vas a dividir entre 0??? :c.\n");
59                 destruirPila(pilaNum);
60                 return 0.0;
61             }
62             res = a / b;
63             break;
64         case '^':
65             res = 1;
66             for (int i = 0; i < (int)b; i++)
67                 res *= a;
68             break;
69     }
70
71     push(pilaNum, &res, sizeof(float));
72 }
73
74 }
75
76 float fin = 0.0;
77 if (pilaVacia(pilaNum) == 0) {
78     fin = *((float*)pilaNum->tope->dato);
79     pop(pilaNum);
80 } else {
81     printf("Error: expresion vacia o invalida.\n");
82 }
83
84 destruirPila(pilaNum);
85 return fin;
86 }

```

Main.c:

main.c es el punto de entrada del programa y se encarga de unir todo el flujo.

Primero declara el arreglo de variables, el contador de variables y el arreglo donde se guardará la expresión. Luego llama a cargarArchivo pasándole la ruta del archivo de entrada para que llene esas estructuras con los datos leídos. Después imprime la expresión infija tal como fue leída del archivo. A continuación, llama a infijaAPostfija para convertir la expresión y obtener la cola postfija, la cual imprime con imprimirCola. Luego pasa esa cola a evaluarPostfija junto con las variables para obtener el resultado numérico, que imprime con dos decimales. Finalmente libera la memoria de la cola con destruirCola y termina el programa retornando cero.

```

1  #include <stdio.h>
2
3  #include "../include/parser.h"
4  #include "../include/evaluator.h"
5  #include "../include/cola.h"
6
7  int main() {
8
9      Variable variables[MAX_VARIABLES];
10
11     int totalVariables;
12
13     char expresion[MAX_EXPR];
14
15     cargarArchivo(
16         "../input/caso1.txt",
17         variables,
18         &totalVariables,
19         expresion
20     );
21
22     printf("Expresion infija:\n");
23     printf("%s\n\n", expresion);
24
25     Cola* postfija =
26         infijaAPostfija(expresion);
27
28     printf("Expresion postfija:\n");
29
30     imprimirCola(postfija);
31
32     float resultado =
33         evaluarPostfija(
34             postfija,
35             variables,
36             totalVariables
37         );
38
39     printf("\nResultado:\n");
40     printf("%.2f\n", resultado);
41
42     destruirCola(postfija);
43
44     return 0;
45 }

```

Ejecución:

```

Compilation terminated.
@ehidanchavez67 →/workspaces/postfija-2-los-muchachos-5 (main) $ cd src
@ehidanchavez67 →/workspaces/postfija-2-los-muchachos-5/src (main) $ gcc pila.c cola.c parser.c evaluator.c main.c -o
programa
@ehidanchavez67 →/workspaces/postfija-2-los-muchachos-5/src (main) $ ./programa
Expresion infija:
(a+b)*c

Expresion postfija:
a b + c *

Resultado:
16.00
@ehidanchavez67 →/workspaces/postfija-2-los-muchachos-5/src (main) $

```

Al salir con éxito la compilación se concluyo que se implemento correctamente la pila y cola ligada para la resolución del probleba infija-postfija.

Dificultades:

No se tuvieron grandes dificultades para la implementación del programa, no obstante, la prueba de autograding no pudo concretarse debido a errores en la sintaxis del run_test, dicho programa no concordaba con los archivos propuestos en .h y .c, pero fuera de eso se completaron los objetivos exitosamente.

Conclusiones:

Al ser una practica en donde se implementaron 2 TDA'S, fue difícil recordar su implementación, ya que cada uno trabaja de forma independiente, conllevando a una reestructuración de conceptos fundamentales para EDA, no obstante fue de gran ayuda la practica en la implementación del programa, ya que cada función o concepto fue visto en clase, lo que permitió bajos porcentajes de desconocimiento respecto al tema, me sorprendió la facilidad con la que las estructuras de datos influyen en conceptos fundamentales de la matemática moderna, y como estas son de gran ayuda a la hora de planteamientos de problemas.

Referencias bibliográficas:

Universidad Nacional Autónoma de México. (s. f.). Práctica 3: Estructuras de Datos y Algoritmos I. Facultad de Ingeniería.
http://lcp02.fib.unam.mx/static/docs/PRACTICAS_EDA1/eda1_p6.pdf.